

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 20%

QUESTIONS:15 Total

Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I M. Vidhya(192412151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.

Sol)

Design a Pseudocode Algorithm for Dataset Preprocessing

Input: Dataset stored in a file containing numerical and categorical values.

Output: Cleaned, normalized, standardized dataset saved into a new file.

Pseudocode

BEGIN

1. Read the dataset from the input file.

DATA ← READ_FILE("input.csv")

2. Convert the data into a DataFrame.

DF ← CONVERT_TO_DATAFRAME(DATA)

3. Check whether the DataFrame is empty.

IF DF is empty THEN

DISPLAY "Dataset is empty"

STOP

END IF

4. Check the structure of the dataset:

DISPLAY column names, data types and missing values.

5. Handle missing values:

FOR each column in DF

IF column is numerical THEN

Replace missing values with the median of that column.

ELSE IF column is categorical THEN

Replace missing values with the mode of that column.

END IF

END FOR

6. Remove duplicate records:

$DF \leftarrow \text{REMOVE_DUPLICATES}(DF)$

7. Standardize categorical values:

FOR each categorical column

Remove leading and trailing spaces.

Convert values into a common format.

Example:

"male", "Male", " MALE" $\rightarrow$ "Male"

END FOR

8. Filter invalid entries:

Remove records containing invalid values.

Example:

$\text{Age} < 0$ or $\text{Age} > 100 \rightarrow$ Remove record.

9. Normalize numerical columns:

FOR each numerical column X

$\text{MIN} \leftarrow \text{MINIMUM}(X)$

$\text{MAX} \leftarrow \text{MAXIMUM}(X)$

IF $\text{MAX} \neq \text{MIN}$ THEN

$X \leftarrow (X - \text{MIN}) / (\text{MAX} - \text{MIN})$

END IF

END FOR

10. Apply indexing:

Reset the DataFrame index after removing
duplicates and invalid records.

11. Validate the cleaned dataset:

Check that:

a) No missing values remain.

- b) No duplicate records remain.
- c) Numerical values are within valid ranges.
- d) Categorical values have a consistent format.

12. Save the cleaned dataset:

```
SAVE DF AS "cleaned_dataset.csv"
```

13. Display "Data preprocessing completed successfully."

END

Explanation

The algorithm first reads the dataset and converts it into a DataFrame. Missing numerical values are replaced using the median, while missing categorical values are replaced using the mode. Duplicate records are removed to avoid repeated information.

Next, categorical values are standardized into a common format, and invalid entries are filtered using predefined constraints. Numerical columns are then normalized using Min-Max normalization, which converts values into the range 0 to 1.

Finally, the index is reset, the cleaned dataset is validated to ensure that no missing values, duplicates, or invalid values remain, and the final dataset is saved to a new file. Array-based and DataFrame/vectorized operations are preferred for efficient processing of large datasets.

2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.

Sol)

Pseudocode Algorithm to Compute Statistical Measures

Input: A large numerical dataset.

Output: Mean, median, variance, and standard deviation of the valid data.

Pseudocode

BEGIN

1. Read the dataset into an array.

```
X ← READ_DATASET("input.csv")
```

2. Validate the dataset.

IF X is empty THEN

 DISPLAY "Dataset is empty"

 STOP

END IF

3. Remove unwanted and invalid values:

$X \leftarrow \text{FILTER}(X, \text{values are not NULL, NaN or infinite})$

4. Check the number of valid values.

$n \leftarrow \text{LENGTH}(X)$

 IF $n = 0$ THEN

 DISPLAY "No valid data available"

 STOP

 END IF

5. Detect extreme outliers using a suitable method

 such as IQR or predefined valid limits.

6. Exclude unwanted extreme outliers:

$X \leftarrow \text{FILTER}(X, \text{values within acceptable range})$

7. Recalculate the number of elements:

$n \leftarrow \text{LENGTH}(X)$

 IF $n = 0$ THEN

 DISPLAY "No valid values after filtering"

 STOP

 END IF

8. Calculate the mean:

$\text{SUM} \leftarrow \text{SUM}(X)$

$\text{MEAN} \leftarrow \text{SUM} / n$

9. Sort the array:

$\text{SORT}(X)$

10. Calculate the median:

```
IF n is odd THEN
    MEDIAN ←  $X[n/2]$ 
ELSE
    MEDIAN ←  $(X[n/2 - 1] + X[n/2]) / 2$ 
END IF
```

11. Calculate variance:

```
VARIANCE ←  $\text{SUM}((X - \text{MEAN})^2) / n$ 
```

12. Calculate standard deviation:

```
STANDARD_DEVIATION ←  $\sqrt{\text{VARIANCE}}$ 
```

13. Display the results:

```
DISPLAY MEAN
DISPLAY MEDIAN
DISPLAY VARIANCE
DISPLAY STANDARD_DEVIATION
```

END

Explanation

The algorithm first reads and validates the dataset and checks whether it contains any data. Invalid values such as NULL, NaN, and infinite values are removed using filtering operations. Extreme outliers can also be identified using the IQR method or predefined limits and excluded when required.

The mean is calculated by dividing the sum of all valid values by the number of values. The array is sorted to calculate the median. Variance is calculated as the average of the squared differences from the mean, and standard deviation is obtained by taking the square root of the variance.

For a large dataset, array-based/vectorized operations such as SUM, filtering, sorting, and aggregation are used because they are more efficient than processing each value individually.

Edge Cases

- Empty array: Display an error and stop.
- All values invalid: No statistical measures are calculated.
- Extreme outliers: Detect and filter them using an appropriate method.
- Single valid value: Mean and median can be calculated, while variance depends on the chosen population/sample formula.

3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.

Sol)

Pseudocode Algorithm to Combine Two or More DataFrames

Input: Two or more DataFrames containing related data and a common key.

Output: A single merged, validated, grouped, and sorted DataFrame.

Pseudocode

BEGIN

1. Read the DataFrames:

DF1 ← READ_FILE("data1.csv")

DF2 ← READ_FILE("data2.csv")

DF3 ← READ_FILE("data3.csv")

2. Validate each DataFrame:

Check whether the DataFrames are empty.

Check whether the common key is present.

Check whether the data types of common keys are the same.

3. Align the indices:

Reset the index of each DataFrame.

4. Remove duplicate records:

DF1 ← REMOVE_DUPLICATES(DF1)

DF2 ← REMOVE_DUPLICATES(DF2)

DF3 ← REMOVE_DUPLICATES(DF3)

5. Merge the DataFrames using the common key:

MERGED ← MERGE(DF1, DF2, ON="CommonKey")

6. Merge the remaining DataFrame:

MERGED ← MERGE(MERGED, DF3, ON="CommonKey")

7. Remove duplicate records from the merged DataFrame:

```
MERGED ← REMOVE_DUPLICATES(MERGED)
```

8. Group the merged data:

```
GROUPED ← GROUP_BY(MERGED, "CommonKey")
```

9. Perform required aggregation:

```
RESULT ← AGGREGATE(GROUPED)
```

10. Sort the result:

```
RESULT ← SORT(RESULT, BY="CommonKey")
```

11. Validate the merged data:

Check that common keys are correctly matched.

Check for unexpected missing records.

Check that important fields are complete.

Check that record counts are consistent.

12. Reset the final index:

```
RESULT ← RESET_INDEX(RESULT)
```

13. Save the final DataFrame:

```
SAVE RESULT AS "merged_dataset.csv"
```

END

Key point:

Merging is performed using the common key, while grouping, aggregation, and sorting organize the final data. Duplicate removal and validation ensure consistency and completeness of the merged records.

4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.

Sol)

Pseudocode Algorithm for Filtering and Sorting a Dataset

Input: Dataset containing numerical and categorical values.

Output: Dataset satisfying all given filtering conditions and arranged in sorted order.

Pseudocode

BEGIN

1. Read the dataset:

$DF \leftarrow \text{READ_FILE}(\text{"input.csv"})$

2. Check whether the dataset is empty.

 IF DF is empty THEN

 DISPLAY "Dataset is empty"

 STOP

 END IF

3. Define the filtering conditions:

$\text{CONDITION1} \leftarrow (\text{Age} \geq 18 \text{ AND } \text{Age} \leq 60)$

$\text{CONDITION2} \leftarrow (\text{Category} = \text{"A"})$

$\text{CONDITION3} \leftarrow (\text{Value} \geq 50 \text{ AND } \text{Value} \leq 500)$

4. Combine all conditions:

$\text{FINAL_CONDITION} \leftarrow \text{CONDITION1 AND}$

CONDITION2 AND

CONDITION3

5. Apply Boolean indexing:

$\text{FILTERED} \leftarrow DF[\text{FINAL_CONDITION}]$

6. Check the filtered result:

 IF FILTERED is empty THEN

 DISPLAY "No records satisfy the conditions"

 STOP

 END IF

7. Sort the filtered data:

```
SORTED ← SORT(FILTERED, BY="Value",  
              ORDER="Ascending")
```

8. Reset the index:

```
SORTED ← RESET_INDEX(SORTED)
```

9. Validate the output:

Verify that every record satisfies all conditions.

Verify that the values are correctly sorted.

10. Save the result:

```
SAVE SORTED AS "filtered_sorted.csv"
```

11. Display the filtered and sorted dataset.

END

Efficiency

For large datasets, Boolean indexing and vectorized logical operations should be used instead of row-by-row checking. Filtering should be performed before sorting, because sorting a smaller dataset requires less computation.

5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

Sol)

Pseudocode Algorithm for Complete Data Analysis Workflow

Input: Raw dataset stored in a file.

Output: Cleaned DataFrame, grouped and summarized results, and plots.

Pseudocode

BEGIN

1. Specify the input file:

```
FILE ← "input.csv"
```

2. Read the file:

```
TRY  
    DATA ← READ_FILE(FILE)  
CATCH FileNotFoundError  
    DISPLAY "Error: File not found"  
    STOP  
CATCH OtherError  
    DISPLAY "Error while reading the file"  
    STOP  
END TRY
```

3. Convert the data into a DataFrame:

```
DF ← CONVERT_TO_DATAFRAME(DATA)
```

4. Inspect the dataset:

```
DISPLAY column names  
DISPLAY data types  
DISPLAY number of rows and columns
```

5. Handle missing and invalid values:

```
Replace or remove missing values.  
Remove invalid records.  
Convert columns to appropriate data types.
```

6. Apply data transformations:

```
Normalize numerical columns if required.  
Standardize categorical values.  
Remove duplicate records.
```

7. Group the data:

```
GROUPED ← GROUP_BY(DF, "Category")
```

8. Perform aggregation:

```
SUMMARY ← AGGREGATE(GROUPED,
```

COUNT,
MEAN,
SUM,
MIN,
MAX)

9. Sort the summarized results:

```
SUMMARY ← SORT(SUMMARY, BY="Mean",  
ORDER="Descending")
```

10. Validate the summarized data:

Check for missing results.

Check that all required groups are present.

Verify aggregation values.

11. Generate plots:

```
CREATE_BAR_PLOT(SUMMARY)
```

```
CREATE_LINE_PLOT(SUMMARY)
```

```
CREATE_HISTOGRAM(Df)
```

12. Add suitable titles, axis labels and legends.

13. Save the results:

```
SAVE SUMMARY AS "summary.csv"
```

SAVE plots as image files.

14. Display:

```
"Data analysis completed successfully."
```

END

Workflow

File Input → DataFrame Conversion → Data Validation → Data Cleaning → Transformation
→ Grouping → Aggregation → Sorting → Visualization → Final Output

This workflow provides a complete process from loading the raw data to generating summarized results and plots, while also handling file errors and data quality issues.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand